Week 8 — User Mode, Syscalls, and IPC

Format: seven daily sections collected into one mastery package

Suggested workload: approximately 21 focused hours, adjustable to mastery

> Keep three layers separate: architecture moves privilege and checks pages; an ABI assigns meaning to registers and frames; kernel policy decides what a valid request may do.

Week 8: one request, six boundaries

Architecture moves privilege; ABI names registers; policy validates effects.

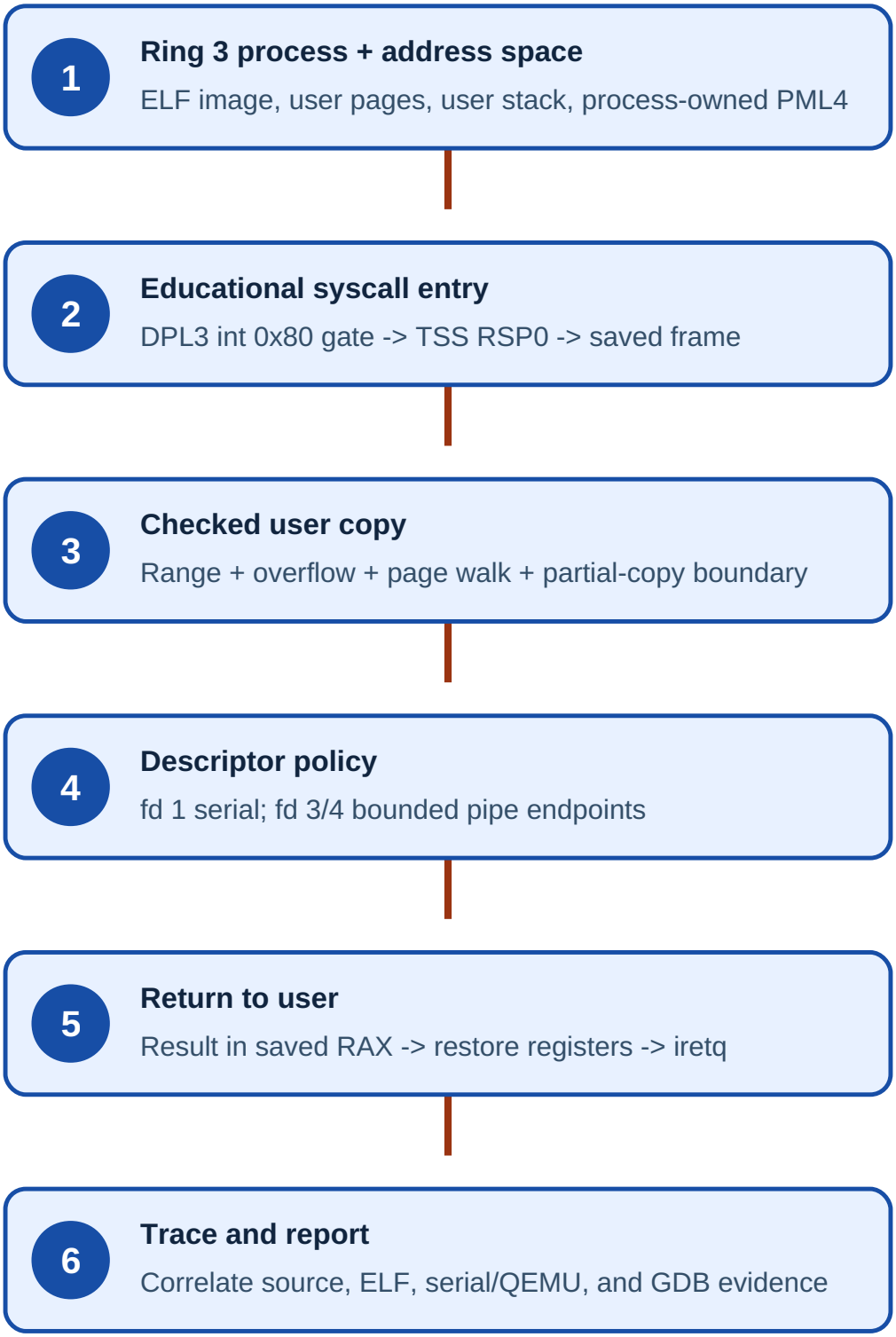


Figure 1. The package roadmap keeps privilege mechanics, ABI meaning, kernel policy, ownership, and evidence connected across the full request path.

This package studies the existing `projects/toyos` Week 8 milestone. Every practical challenge inspects or runs that canonical project; no challenge forks or patches the kernel. Each wrapper follows:

```
purpose → prediction → evidence → explanation → limits
```

ToyOS Week 8 has exactly one ring-3 process. It owns a PML4, user mappings, a four-page user stack, a 16 KiB kernel stack, PID 1, and a small descriptor convention. It does not implement fork/exec, teardown, demand paging, blocking, shared-memory mappings, or signals. Its 64-byte global pipe is the bounded IPC mechanism available in the current project.

Daily roadmap

Day 1 — Ring 3 entry and the architecture lens

Audit user descriptors, TSS `RSP0`, the DPL3 gate, and the crafted `iretq` frame. Distinguish CPU privilege rules from ToyOS choices.

- Lesson: day-001-ring3-gdt-tss-iret.md
- Challenge: `challenges/day-001-ring3-entry.sh`

Day 2 — User address spaces and U/S permissions

Follow the process-owned PML4, copied kernel mappings, user propagation through all page-table levels, CR3 activation, and the isolation limits.

- Lesson: day-002-user-address-spaces.md
- Challenge: `challenges/day-002-address-space.sh`

Day 3 — Embedded ELF loading

Validate the ELF64 header and program-header table, load `PT_LOAD` segments, map the user stack, and identify validation and rollback limits.

- Lesson: day-003-embedded-elf-loader.md
- Challenge: `challenges/day-003-elf-loader.sh`

Day 4 — `int 0x80` syscall ABI

Trace the DPL3 interrupt gate, saved frame, dispatch registers, result in `RAX`, and `iretq` return. Explain explicitly why this is not a `SYSCALL`/`SYSRET` implementation.

- Lesson: day-004-int80-syscall-abi.md
- Challenge: `challenges/day-004-syscall-path.sh`

Day 5 — Safe user copying

Walk and validate user translations page by page, enforce the 4096-byte copy bound, and inspect the deterministic invalid-pointer profile.

- Lesson: day-005-safe-user-copying.md
- Challenge: `challenges/day-005-user-pointer.sh`

Day 6 — Process, descriptors, and bounded IPC

Account for process and descriptor state, the 64-byte pipe, and the distinction between implemented pipe IPC and conceptual shared-memory/signal boundaries.

- Lesson: day-006-process-fd-ipc.md
- Challenge: `challenges/day-006-pipe-ipc.sh`

Day 7 — Integrated trace, diagnosis, and comparison

Build an evidence report from user instruction through syscall and return, interpret both Week 8 injected profiles, and compare ToyOS with xv6 and Linux.

- Lesson: day-007-trace-debug-comparison.md
- Challenge: `challenges/day-007-user-kernel-report.sh`

Integrated checkpoint

Narrate this chain without skipping an owner:

```
embedded ELF bytes → validated PT_LOAD → physical frames → user PTEs
→ CR3 activation → TSS RSP0 → iretq to CPL3 → int 0x80
→ hardware stack transition → saved interrupt_frame → syscall policy
→ checked copy / pipe state → RAX result → iretq to user
```

At every arrow name:

1. the architecture rule, such as CPL/DPL/RPL, page U/S, or interrupt entry;
2. the ABI rule, such as syscall numbers and register arguments;
3. the ToyOS policy, such as the 256-byte I/O and 64-byte pipe bounds;
4. the mutable owner and its lifetime;
5. the exact source, ELF, or QEMU evidence supporting the claim.

Debugging discipline

Kernel debugging starts with artifact identity. Use the exact `build/week8/toyos.elf` or fault-profile ELF that produced the observation. The canonical `gdb/week8.gdb` stops at `kernel_main`, `process_init`, and `syscall_dispatch`. Useful follow-up observations include the saved frame, `CR3`, `CR2`, current `RSP`, `CS`, `SS`, the IDT gate, and page-table entries. Do not claim a GDB observation merely because a command is suggested.

The normal profile exits through QEMU with host status 33 and marker `WEEK8: PASS`. The bad-user-pointer profile checks marker `EXPECTED-FAULT: user-pointer PASS` and host status 43. The unknown-syscall profile checks marker `EXPECTED-FAULT: syscall PASS` and host status 45. These are controlled rejection tests, not crashes.

Reference discipline

Current xv6-riscv comparison landmarks commonly include `kernel/proc.c`, `kernel/vm.c`, `kernel/exec.c`, `kernel/trampoline.S`, `kernel/trap.c`, `kernel/syscall.c`, `kernel/pipe.c`, and `kernel/signal`-equivalent absence; verify the revision you cite. Linux landmarks include `arch/x86/entry/`, `arch/x86/mm/`, `fs/`, `ipc/`, `kernel/signal.c`, and `mm/`; inspect a current tree before making line-level claims.

Do not auto-fill mastery, confidence, hours, or completion.

Day 1 — Ring 3, the GDT, TSS, and `iretq`

Target time: approximately 3 hours

> “Enter user mode” is not one instruction. It is a chain of descriptor, stack, frame, mapping, and return assumptions that must agree.

1. Begin with privilege, not process vocabulary

x86-64 exposes four privilege levels, but mainstream kernels normally use CPL0 for the kernel and CPL3 for applications. CPL is encoded in the low two bits of `CS`; selectors also carry a requested privilege level, and segment descriptors carry a descriptor privilege level. The processor applies rules involving CPL, RPL, and DPL when software loads selectors or invokes gates.

Those checks are architecture mechanism. Saying “PID 1 may write to serial” is policy. Saying syscall 1 uses `RDI`, `RSI`, and `RDX` is ABI. Keep those sentences separate while reading ToyOS.

ToyOS adds two user descriptors in `projects/toyos/src/arch.c`. GDT entry 3 is user data and entry 4 is user code. Their DPL is 3. The selectors used by `enter_user` are `0x1b` for user data and `0x23` for user code: the low bits request ring 3. The numerical selector values are a consequence of this GDT layout, not universal x86 constants.

2. Understand why a TSS still matters

Long mode does not use the old hardware task-switching model for ordinary process scheduling, but a 64-bit TSS remains essential. It stores privilege stack pointers `RSP0`–`RSP2`, interrupt-stack-table pointers, and an I/O-map boundary.

At boot, `gdt_init` creates a two-slot 64-bit TSS descriptor in GDT entries 5 and 6, initializes `tss.rsp[0]` to the static interrupt stack, points IST1 at the same initial top, and executes `ltr` with selector `0x28`. Week 6 used that infrastructure for selected exceptions. Week 8 updates `RSP0` before entering user mode:

```
process kernel-stack allocation
→ align top to 16 bytes
→ arch_set_kernel_stack
→ tss.rsp[0] = kernel_top
```

The process owns the 16 KiB allocation for kernel lifetime in this milestone. There is no process teardown, so there is no reclamation path. The TSS does not own the bytes; it holds the address the CPU will use on a ring-3-to-ring-0 gate transition.

)escriptors name privilege; TSS names entry stac

Selector low bits request ring 3; descriptor DPL authorizes it.

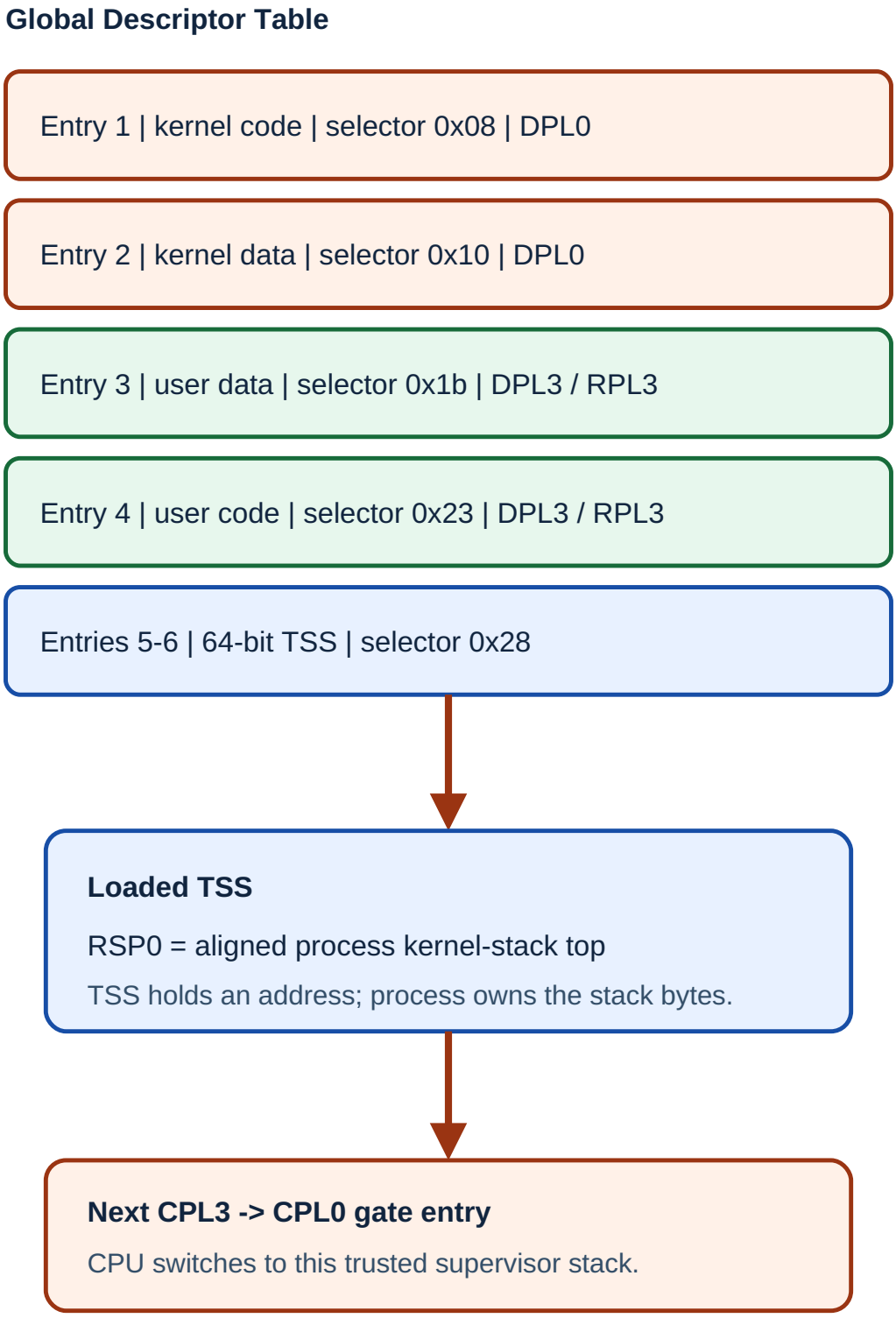


Figure 1. Descriptor privilege authorizes ring-3 selectors, while the loaded TSS supplies the trusted kernel-stack address for the next privilege entry.

3. Read the initial `iretq` frame bottom-up

`process_start` first activates the process PML4. It then updates TSS `RSP0` and calls `enter_user(entry, user_stack)`. In `projects/toyos/src/user_entry.S`, `enter_user` disables interrupts while constructing the frame, loads user data selectors into `DS` and `ES`, and pushes:

```
SS = 0x1b
RSP = user stack top
RFLAGS with IF set
CS = 0x23
RIP = validated ELF entry
```

Because stacks grow downward, the final push leaves `RIP` at the top. `iretq` consumes this privilege-return frame and changes CPL from 0 to 3. The instruction is being used as a controlled first entry even though there was no earlier user interrupt to return from.

The assembly ORs bit 9 into the saved flags, enabling maskable interrupts after the transition. It does not execute `sti` in ring 3. The kernel controls the future `RFLAGS` value through the crafted frame.

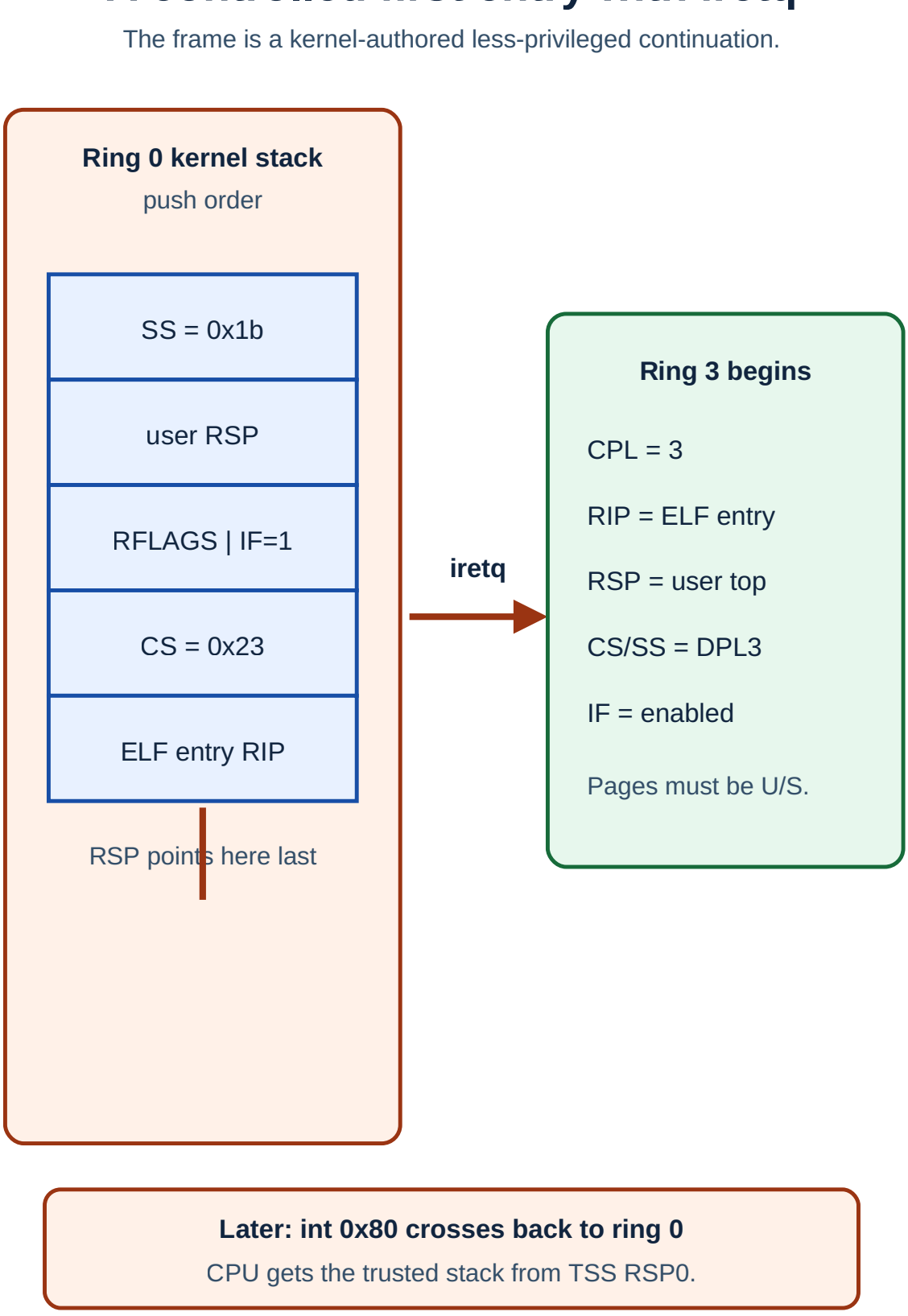


Figure 2. The kernel-authored `iretq` frame establishes the initial ring-3 continuation; a later `int 0x80` returns through TSS `RSP0`.

4. Account for every prerequisite

`iretq` succeeds only if several independent contracts already hold:

- `CS` and `SS` selectors name present, suitable DPL3 descriptors.
- `RIP` and `RSP` are canonical user addresses.
- the process page tables map the instruction and stack with U/S set at every level of the walk;
- the instruction bytes are executable under the current paging policy;
- the stack has writable user pages below `USER_STACK_TOP`;
- TSS `RSP0` names writable supervisor memory for the next privilege entry.

ToyOS validates enough of these for its embedded image, but not all possible hostile ELF properties. For example, `load_elf` bounds load segments yet does not separately prove that `e_entry` lies within a mapped executable segment. The teaching mapper also does not enforce NX or W^X, so all load segments are writable.

5. The syscall gate completes the round trip

`arch_init` installs IDT vector 128 with attributes `0xee`. This is a present 64-bit interrupt gate with DPL3. DPL3 is what allows a CPL3 `int $0x80` instruction to select it; ordinary exception gates remain DPL0 unless the architecture gives them other semantics. As an interrupt gate, it clears IF on entry.

When `int $0x80` crosses privilege, the CPU obtains the ring-0 stack pointer from the loaded TSS, switches stacks, and records the old user `SS`, `RSP`, `RFLAGS`, `CS`, and `RIP`. The assembly stub adds normalized vector and error slots and saves general registers. That final in-memory shape matches `struct interrupt_frame`.

This stack switch is the concrete reason `RSP0` cannot be stale or shared carelessly. In a multi-process kernel, every scheduling transition must make the TSS point to the kernel stack belonging to the process that can next enter. ToyOS has one process and avoids that coordination problem.

6. Debug the transition with architecture evidence

Static evidence can prove descriptor constants, push order, symbols, and instructions. Runtime evidence is needed to prove a selected path executed. For an interactive session, build and pause `debug-week8`, attach using the canonical Week 8 GDB script, then inspect:

```
before enter_user: CR3, tss.rsp[0], entry, user_stack
at syscall_dispatch: CS/SS saved in frame, saved user RIP/RSP, live kernel RSP
before iretq: restored register values and remaining hardware frame
```

State predictions before running. At dispatch, live CPL is 0 while `frame->cs` should describe the interrupted ring-3 context. Do not confuse the current debugger register `CS` with saved `frame->cs`.

7. Architecture comparison

RISC-V commonly enters user mode with an `sret` after setting `sepc`, `sstatus`, and an appropriate user page table; traps use `stvec`, `sscratch`, and software-managed trap-entry conventions. AArch64 uses exception levels, `ELR_EL1`, `SPSR_EL1`, vector tables, and `eret`. These are architecture counterparts, not alternative names for a TSS or x86 selector.

The transferable idea is the invariant set: establish a less-privileged continuation, provide a trusted entry stack, define saved-state ownership, and validate the return state.

Practical challenge

Use `challenges/day-001-ring3-entry.sh` from the repository root.

Purpose: connect GDT/TSS construction, the user IDT gate, and the initial `iretq` frame to the exact Week 8 artifact.

Predict: identify selector values, push order, normal host status 33, and the ring3/TSS/int80 self-test marker.

Evidence: the wrapper displays canonical source contracts, builds and checks Week 8, and disassembles `enter_user` and the syscall stub from the exact ELF.

Mastery prompts

Explain why the TSS is used even though ToyOS does not use hardware task switching. Then classify each of these as architecture, ABI, or policy: DPL3 on vector 128; syscall number 1; maximum I/O length 256; `RAX` return value; the CPU loading `RSP0`; PID 1.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Ring3 Entry

Format: shell lab

What you will do: Audit GDT/TSS state and the canonical iretq transition into ring 3.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-001-ring3-entry.sh
```

- View lab source on GitHub

Day 2 — User Address Spaces, U/S Permissions, and Kernel Sharing

Target time: approximately 3 hours

> A private root is not automatically a private system. Isolation comes from every reachable entry, every shared mapping, and every kernel access path.

1. Follow ownership from the root

`process_init` calls `address_space_create` and stores the returned pointer in `init_process.root`. `frame_alloc` supplies one frame for the new PML4 and one for a replacement first PDPT. Allocator frames are identity-mapped in this teaching kernel, so physical addresses can also be dereferenced as kernel virtual pointers.

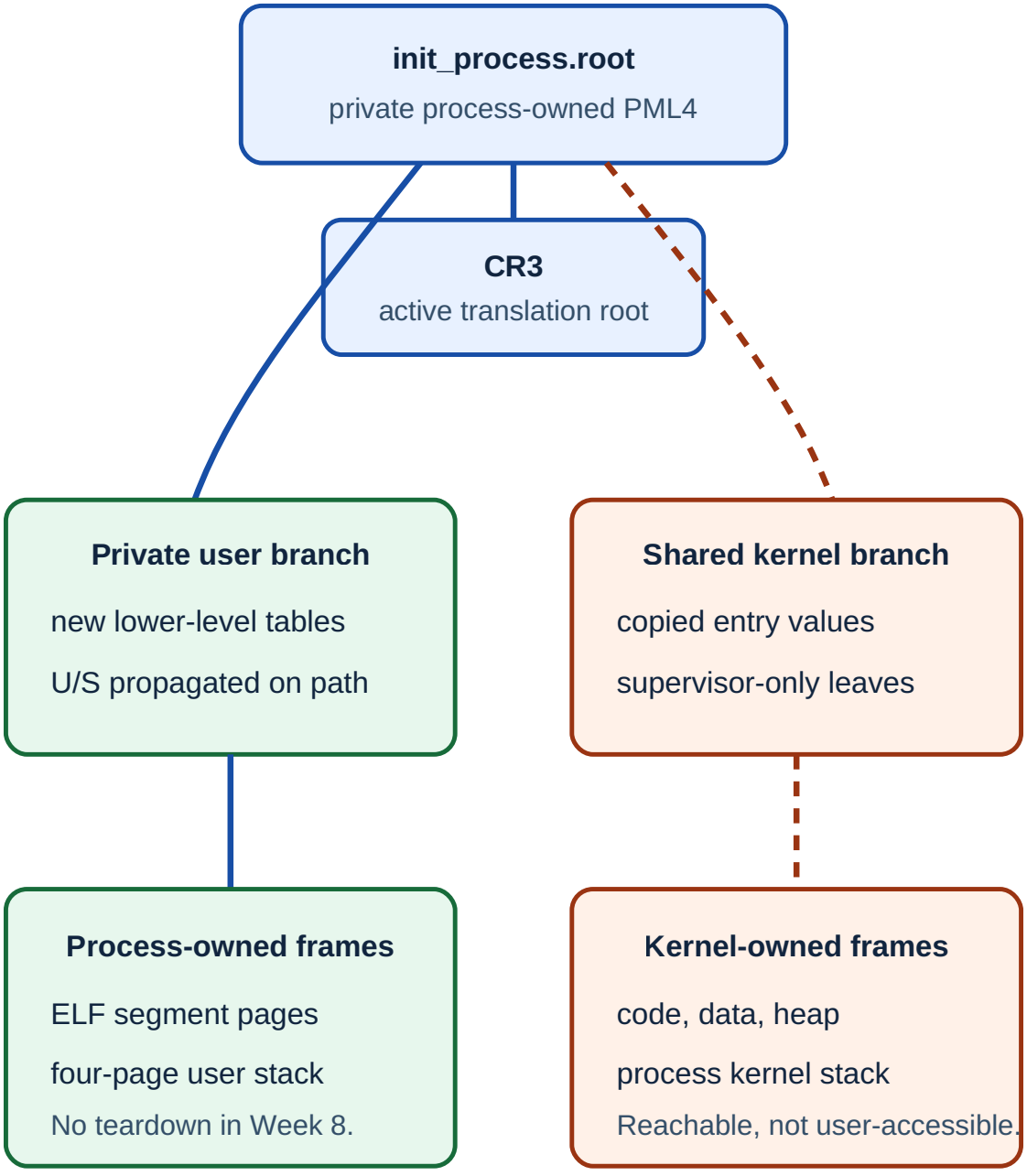
The process owns these paging structures for the rest of the boot. There is no reference count, teardown, or rollback. If the second allocation fails, the first frame is leaked. That is an honest consequence of the compact implementation, not a property of paging.

The function copies all 512 kernel PML4 entries and all 512 entries from the bootstrap root's first PDPT. It then replaces `root[0]` with the new PDPT, marked present and writable but supervisor-only. Thus the process receives the existing low identity mappings needed by kernel code and data, while gaining private lower-level structures beneath its first PML4 slot.

“Private PML4” is accurate. “Completely private address space” needs qualification: many kernel mappings and their underlying physical frames are shared. The copied entries are values pointing at common tables. ToyOS relies on U/S permissions to keep CPL3 out.

Private root, deliberately shared kernel reach

Ownership differs for roots, tables, mappings, and physical frames.



A new PML4 alone is not isolation.

Every reachable entry and permission must support the claim.

Figure 1. Address-space ownership is layered: the process owns its root and user branches, while copied entries retain shared kernel reachability.

2. Apply the U/S rule at every level

For a user access to a 4 KiB page, the U/S bit must permit user access through every paging-structure entry in the walk. Setting `PTE_USER` only on the leaf is insufficient if a PML4E, PDPTE, or PDE remains supervisor-only.

ToyOS handles this in `next_table`. When a user mapping creates an intermediate table, that entry includes `flags & PTE_USER`. When the table already exists in Week 8, the code ORs the user bit into the parent entry. `map_in_root` repeats this at each level and installs the leaf with requested low flags plus present.

This propagation has an important review question: what else is reachable through an intermediate table whose parent was widened to user? Isolation still depends on each leaf's U/S bit, but broad parent changes should be audited. A mature kernel deliberately partitions user and kernel ranges and defines which upper-level structures may be shared.

Ring 3 permission is an all-level AND

Present and U/S must permit the access through the complete walk.

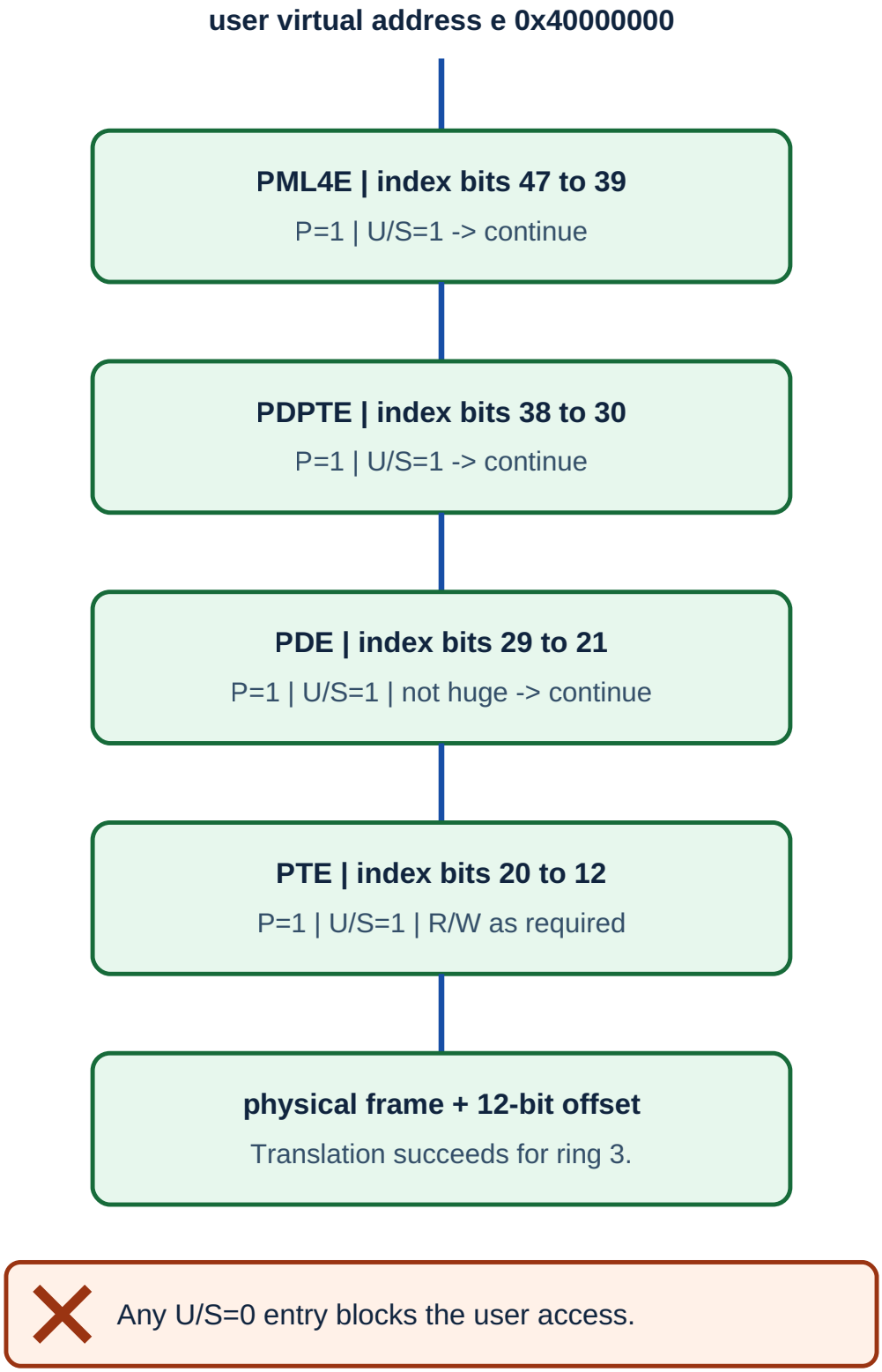


Figure 2. A ring-3 translation succeeds only when every paging level is present and user-accessible; any supervisor-only entry blocks it.

3. Separate virtual ownership from physical ownership

The address-space root belongs to the process, while copied kernel entries refer to shared paging structures and mappings. Each ELF and user-stack leaf gets a newly allocated physical frame. The process conceptually owns those frames, although ToyOS has no structure that can enumerate and release them.

This yields several distinct claims:

```
virtual range owner: process policy
page-table frame owner: process address-space implementation
user data frame owner: loaded process image or stack
kernel mapping owner: kernel, shared into the process root
active translation owner: CPU selected by CR3
```

Do not reduce all five to “the process owns memory.” Debugging leaks, use-after-free, and cross-process exposure requires the narrower vocabulary.

4. Activate the root and understand TLB scope

`address_space_activate` saves the root in `active_root` and moves its address to `CR3`. Loading `CR3` changes the hardware translation context and normally invalidates relevant non-global translations. ToyOS does not use PCIDs, global-page policy, multiple CPUs, or TLB shootdowns.

The software variable `active_root` is not authoritative hardware state. It exists so `copy_from_user` and `copy_to_user` can walk the same root the single-CPU kernel expects to be active. If future scheduling changed `CR3` without changing `active_root`, safe-copy validation could inspect one address space and execute against another. Their synchronized update is an invariant.

`map_in_root` always executes `invlpg` after installing a leaf, even when editing a non-active root. This is harmless for the current demonstration but not a complete address-space-tagged invalidation design.

5. Locate the user image and stack

The linker places the embedded program near `0x40000000`. The loader accepts segments beginning at or above that address and below the reserved stack range. The stack top is `0x40020000`; four pages are mapped immediately below it with user and write permission. The initial stack pointer equals the top, so the first push enters the highest mapped page.

There is no guard page policy explicitly tested, no stack growth, no argument/environment block, and no randomized layout. The gap between image and stack is a fixed teaching layout, not ASLR.

The kernel's heap also begins at `0x50000000`, outside the accepted user segment range, and its mappings are supervisor-only. The same numerical address can have different meanings under different roots in a general kernel; in this project, shared low kernel mappings constrain that freedom.

6. Use page faults as permission evidence

An x86 page-fault error code distinguishes not-present from protection and user from supervisor access. A CPL3 access to a present supervisor-only page should produce a protection fault with the user bit set in the error code. ToyOS does not include a dedicated Week 8 profile that intentionally performs that user instruction, so do not claim it was dynamically tested.

The invalid-user-pointer profile is different: while in the kernel, it asks the software page-table walker to translate `0x1234`. The walker rejects the missing user-accessible chain and returns `false`; no architectural #PF is required. This is preferable for syscall pointer validation because the kernel controls the failure and returns an error rather than blindly dereferencing.

7. Compare designs without erasing architecture

xv6-riscv creates per-process page tables, maps user pages, and uses a trampoline/trapframe arrangement for transitions. RISC-V PTE_U and `satp` play related roles, but the walk shape and trap machinery differ.

Linux x86-64 has richer split-address-space policy, demand paging, VMAs, copy-on-write, PCID-aware TLB management, hardened user-access routines, and SMP synchronization. Configuration and mitigation choices can change how kernel mappings are exposed in user page tables. Inspect a current kernel rather than treating one historical layout as permanent.

ToyOS demonstrates the minimum causal chain: create a root, preserve kernel execution reachability, map user leaves with user permission throughout the walk, load CR3, and enter CPL3. It does not prove adversarial isolation.

Practical challenge

Use `challenges/day-002-address-space.sh` from the repository root.

Purpose: audit private paging-structure ownership, shared kernel mappings, U/S propagation, and CR3 activation.

Predict: the normal profile will emit the private-PML4/ELF marker and exit with status 33; user leaves require U/S at every level.

Evidence: the wrapper shows current source, checks the canonical VM, and prints symbols/disassembly from `build/week8/toyos.elf`.

Mastery prompts

Draw one user and one kernel translation from the process root. Mark each table/frame as private or shared. Then explain why “new PML4” alone proves neither confidentiality nor correct kernel reachability.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Address Space

Format: shell lab

What you will do: Inspect the process PML4, shared kernel mappings, user permissions, and CR3.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-002-address-space.sh
```

- View lab source on GitHub

Day 3 — Embedded ELF Validation, `PT_LOAD`, Entry, and Stack

Target time: approximately 3 hours

> An executable loader is a parser at a security boundary. “The compiler made this file” does not remove the need to validate lengths, arithmetic, and destination policy.

1. Establish where the bytes come from

Week 8 does not read a filesystem. The Makefile first links `build/week8/user.elf` from `user_program.o` with `user.ld`. It then assembles `src/user_image.S` with an absolute `USER_IMAGE` path so the ELF bytes become part of a kernel object. Linker symbols `user_elf_start` and `user_elf_end` bound that embedded byte array.

This arrangement makes the lesson deterministic and bootstraps before Week 9 VFS work. It is still a real ELF64 image with an ELF header and program headers. The loader parses those bytes rather than relying on kernel section names.

Artifact identity matters. `readelf -h -l build/week8/user.elf` describes the user image; `build/week8/toyos.elf` is the kernel containing it. Do not use one symbol table to explain the other.

2. Validate before pointer arithmetic

`load_elf` first computes `image_size` from the linker bounds and requires at least one complete `elf_header`. It validates:

- magic `0x7f 'E' 'L' 'F'`;
- class 2, meaning ELF64;
- data 1, meaning little-endian;
- machine 62, meaning x86-64;
- exact program-header entry size;
- a nonzero program-header count;
- the complete program-header table lies within the embedded image.

The helper `range_inside(offset, length, total)` uses subtraction after checking `offset <= total`, avoiding the classic `offset + length` overflow test. That pattern is worth carrying into every binary parser.

There are limits. Multiplication of `phnum * sizeof(program_header)` occurs after casts to `size_t` without an explicit multiplication-overflow check. `e_type`, ELF version, OS ABI, alignment, and header-size fields are not fully validated. The image is trusted build output, but the code should not be described as a hardened general ELF loader.

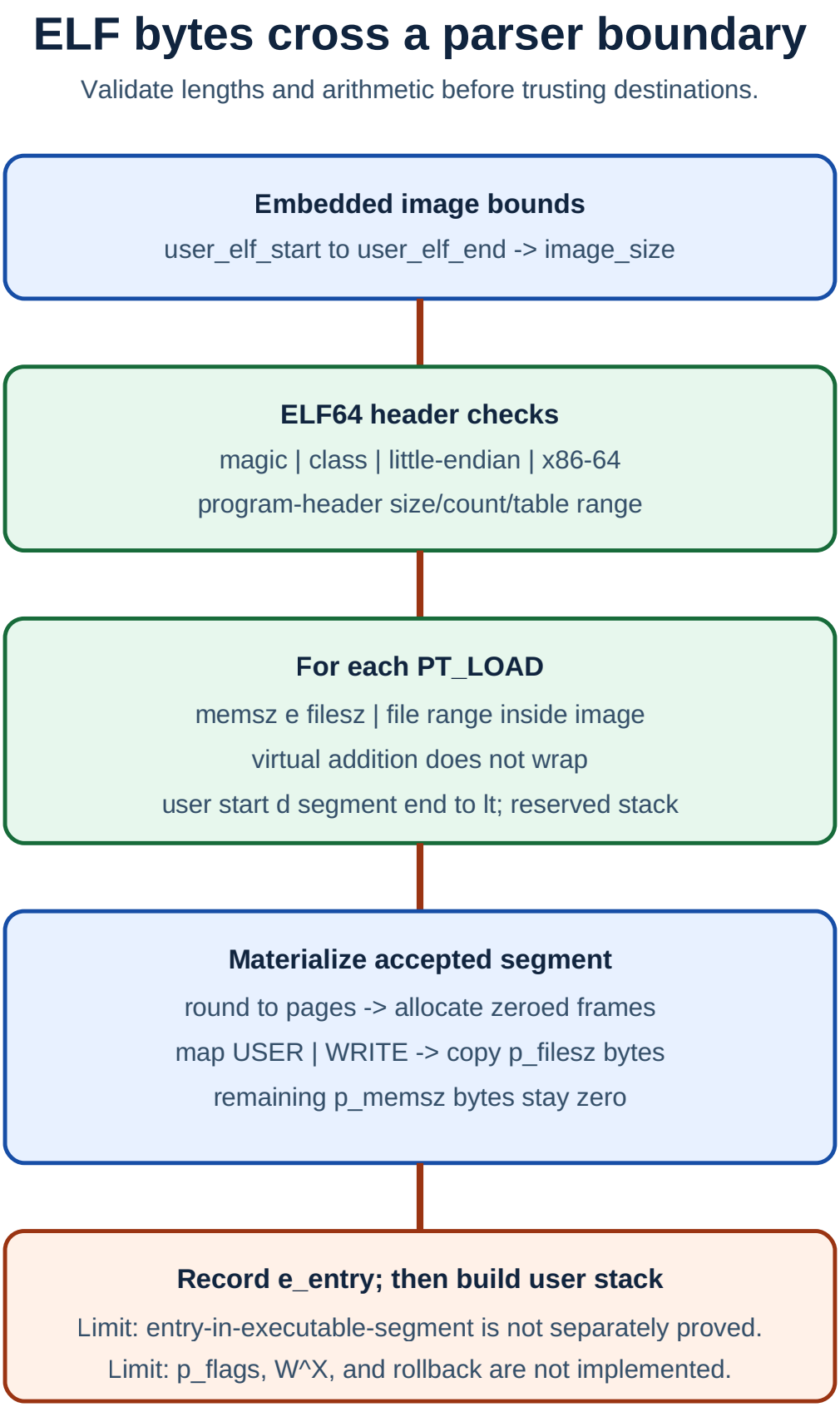


Figure 1. The loader treats the embedded ELF as bounded input: validate structure and arithmetic before materializing any segment.

3. Treat program headers as the load contract

The loader iterates `header->phnum` entries and skips anything not `PT_LOAD`. For each loadable segment it checks:

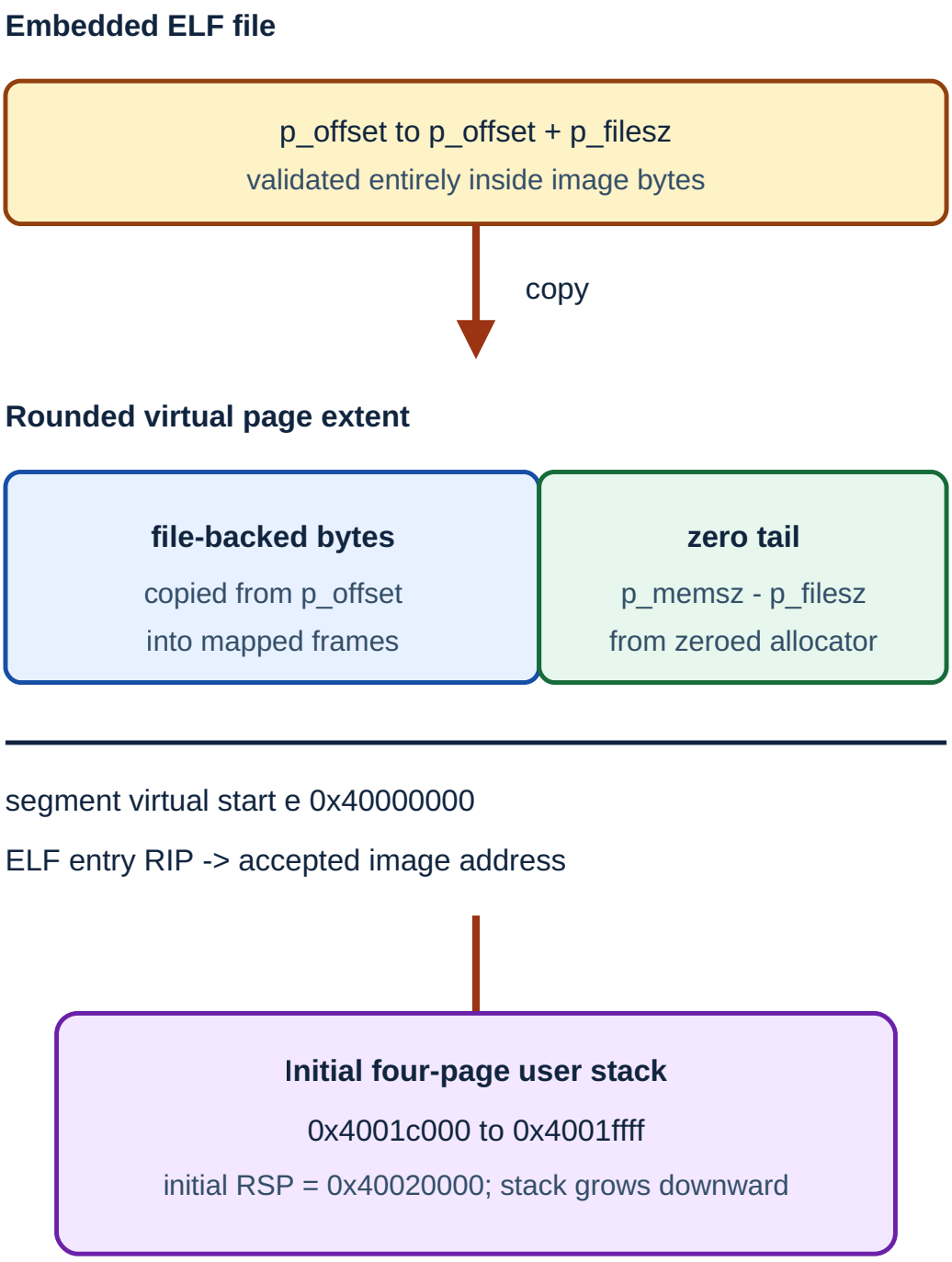
- `memory_size >= file_size`, so initialized bytes fit in memory extent;
- file offset and file size remain within the image;
- virtual address is at least `0x40000000`;
- virtual-address addition does not wrap;
- the end stays below the four-page stack region.

This separates file extent from memory extent. ELF permits `p_memsz` to exceed `p_filesz`; the difference usually represents zero-initialized data. `frame_alloc` zeroes every new frame, so bytes not copied from the file begin as zero. That implicit dependency should be documented whenever allocator behavior changes.

The loader rounds the segment start down and end up to page boundaries, allocates one frame per page, and maps every leaf `PTE_USER | PTE_WRITE`. It then walks the newly built page tables manually for each file byte and copies into the corresponding physical frame through the identity map.

One PT_LOAD becomes owned user memory

File extent and memory extent are related, but not identical.



All mapped leaves are USER | WRITE in this teaching loader.

Figure 2. Each accepted `PT_LOAD` separates file extent from memory extent; zeroed frames supply the tail before entry begins on the four-page stack.

4. Identify policy hidden in compact code

The mapper ignores ELF `p_flags`: text and read-only data become writable, and NX is not enforced. This violates W^X expectations of hardened systems. It is an explicit project limit, not an x86 limitation.

Overlapping `PT_LOAD` page ranges would cause the second `map_page_in` to reject an already-present leaf. There is no rollback, so prior frames and tables leak when loading fails. The loader also does not track which frames belong to which segment.

The byte-at-a-time copy is simple but inefficient. A production loader would copy runs, carefully handle partial first and last pages, zero BSS, apply permissions after relocation/loading, and unwind allocations on failure.

Most importantly, the code stores `header->entry` without proving it lies in a loaded segment or carries intended execute permission. In this canonical image it does. As a parser claim, however, segment-range validation does not automatically validate the entry point.

5. Build the initial user stack

After loading segments, `process_init` allocates four frames and maps:

```
0x4001c000 through 0x4001ffff
```

with `USER | WRITE`, then records `USER_STACK_TOP` as `0x40020000`. The stack grows downward. It starts empty: no `argc`, `argv`, `envp`, auxiliary vector, return address, or randomized gap is installed. The assembly user program is written to need none of them.

There is no explicit guard-page test. The next page below the mapped range is expected to be absent unless an ELF segment occupied it, which the segment upper bound prevents. There is no automatic growth; an access below the four pages faults.

The kernel stack is separate. It is allocated from the supervisor heap and is never mapped user-accessible. The TSS points to its aligned top for privilege entry. User RSP and kernel RSP are different resources with different owners and trust levels.

6. Inspect ELF evidence precisely

Use `readelf` to answer concrete questions:

- What is the ELF entry address?
- How many program headers exist?
- Which headers are `LOAD`?
- What are each header's file size, memory size, flags, and alignment?
- Do virtual ranges stay below the stack reservation?

Use `nm -n build/week8/user.elf` or `objdump -d` to associate `_user_start` with its instructions. Use the kernel ELF to locate `load_elf`, `process_init`, and embedded-image symbols.

The wrapper checks runtime markers too, but a successful boot exercises only the canonical ELF. It does not fuzz malformed headers. Static inspection of validation branches supports the control logic; it is not equivalent to executing every rejection.

7. Compare executable loading models

xv6 `exec` reads an executable through its filesystem, creates/replaces a process page table, loads segments, builds a user stack containing arguments, and commits the new image after successful preparation. Its executable format and architecture details should be checked against the current tree.

Linux `execve` is a large policy boundary involving binary-format handlers, credentials, VMAs, demand paging, argument/environment limits, auxiliary vectors, interpreter loading, ASLR, permission checks, and cleanup. ELF program headers are still central, but the lifecycle and security model are far richer.

ToyOS isolates one idea: validated file ranges become owned physical pages and user translations before a controlled entry. Preserve that causal model while being explicit about absent commit/rollback and protection policy.

Practical challenge

Use `challenges/day-003-elf-loader.sh` from the repository root.

Purpose: correlate loader checks with the exact embedded user ELF and normal execution.

Predict: identify the entry, `PT_LOAD` ranges, four-page stack interval, normal host status 33, and process-loaded marker.

Evidence: the wrapper builds Week 8, runs `readelf` on the exact user artifact, displays loader contracts, checks QEMU, and prints loader symbols.

Mastery prompts

For each accepted segment, record file extent, memory extent, rounded page extent, allocated frame owner, PTE permissions, and failure cleanup. Then list three malformed ELF cases rejected by current code and three that need stronger validation.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Elf Loader

Format: shell lab

What you will do: Correlate loader validation with the exact embedded ELF64 user image.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-003-elf-loader.sh
```

- View lab source on GitHub

Day 4 — The `int 0x80` Syscall ABI, Saved Frame, and Return

Target time: approximately 3 hours

> A syscall is an untrusted register packet crossing a privilege boundary. Entry mechanism, register ABI, dispatch policy, and return validation are related but distinct.

1. Write down ToyOS's ABI before tracing code

The user program places a syscall number in `RAX` and arguments in `RDI`, `RSI`, and `RDX`, then executes `int $0x80`. The kernel writes the signed result back to the saved `RAX`. Current numbers are:

```
1 write(fd, user_buffer, length)
2 getpid()
3 yield()
4 exit(status)
5 pipe(user_fds)
6 read(fd, user_buffer, length)
```

Number 7 exists only for the Week 9 build. Unknown numbers return `-38`. Pointer failures use `-14`; oversized reads use `-22`; pipe capacity failure returns `-1`. These resemble Linux errno values in places but are a local teaching ABI, not compatibility with Linux x86-64 or historical i386 `int 0x80`.

The architecture does not know these meanings. It knows only that vector 128 selects an allowed gate.

2. Follow gate installation

`arch_init` calls:

```
set_idt_gate(128, syscall_int_stub, 0, 0xee)
```

The selector is the ring-0 code selector `0x08`. IST is zero, so a privilege-changing entry uses the TSS `RSP0` rather than an IST slot. Attributes `0xee` describe a present DPL3 interrupt gate. DPL3 is required because software interrupt privilege checks would reject a ring-3 caller invoking a DPL0-only gate.

An interrupt gate clears IF during entry. The common path never re-enables it before return, so syscalls execute with maskable interrupts disabled in this compact kernel. That is a policy/performance limitation and also simplifies single-core state reasoning. It is not an inherent requirement for syscall handlers.

3. Reconstruct the saved frame

On a ring-3-to-ring-0 `int`, the CPU switches to the TSS `RSP0` stack and pushes the old user stack selector and pointer, flags, code selector, and continuation RIP. `syscall_int_stub` then pushes a synthetic zero error code and vector 128. `isr_common` saves:

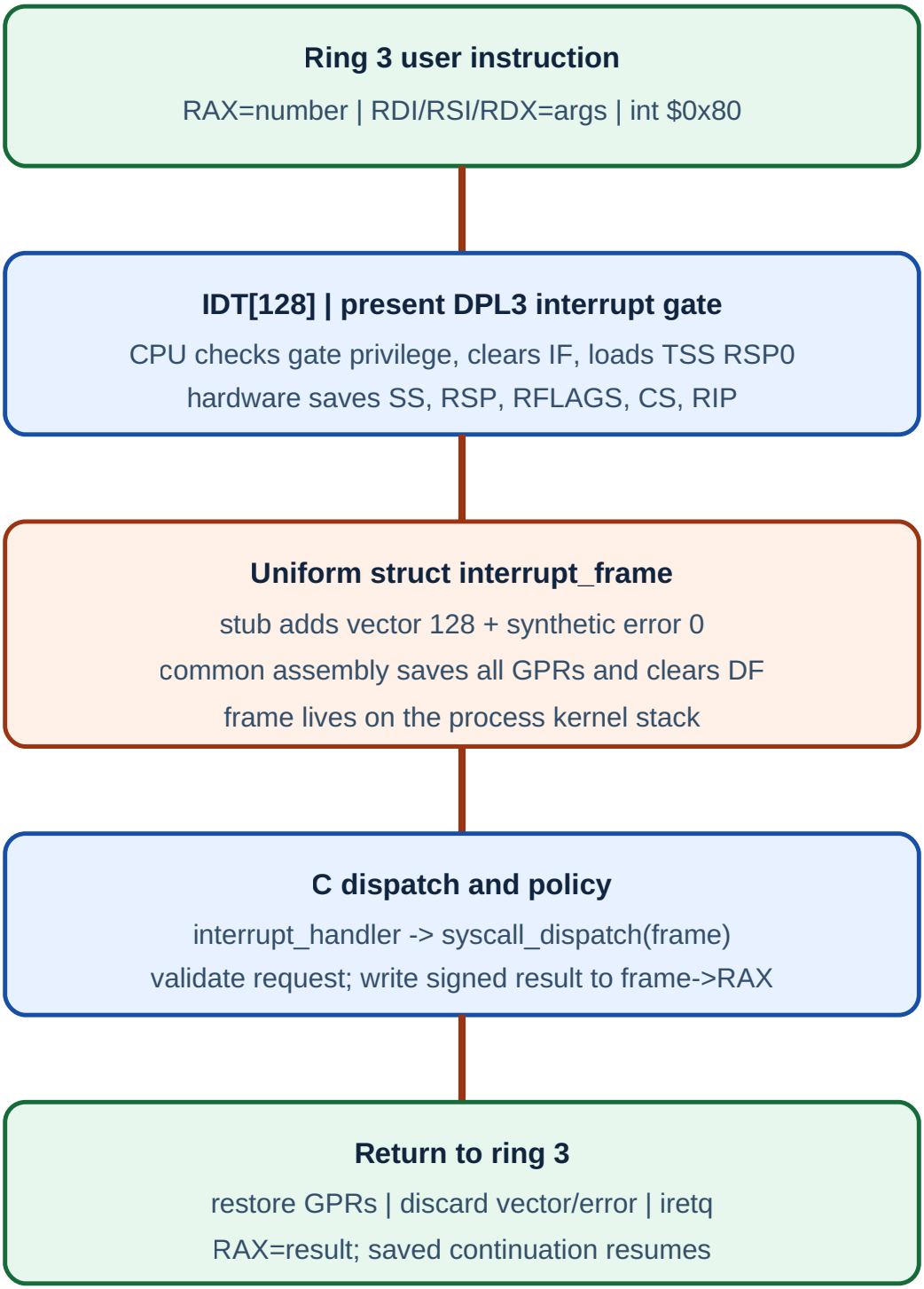
```
RAX RBX RCX RDX RBP RSI RDI R8 ... R15
```

in an order chosen so the final stack pointer matches the field order in `struct interrupt_frame`. It clears direction flag before calling C, moves the frame pointer to `RDI`, and calls `interrupt_handler`.

The frame is owned by the active kernel stack for the duration of entry. It is not copied into the process structure. `syscall_dispatch` may update `frame->rax`; after the C call, assembly pops that modified value into live `RAX`, discards vector/error, and executes `iretq`.

Educational syscall path: DPL3 int 0x80

IDT interrupt gate entry and iretq return--not SYSCALL/SYSRET.



No fast-syscall MSRs, swapgs, SYSCALL, or SYSRET.

Figure 1. ToyOS uses a DPL3 interrupt gate to obtain a trusted stack, normalize a C-visible frame, dispatch, and return through `iretq`.

Frame layout is an ABI between assembly and C. The syscall register convention is a second ABI between user and kernel. Mixing those two layers makes debugging offsets needlessly hard.

4. Dispatch and return

`interrupt_handler` checks vector 128 and calls `syscall_dispatch`. The dispatcher treats every register value as untrusted. It bounds I/O before copying, distinguishes known descriptors, and leaves `result` initialized to unknown-syscall.

The ordinary return path writes `result` into saved `RAX`. The common stub restores all registers. Therefore every register except `RAX` returns with its saved value under the current implementation. This is stronger than many syscall ABIs promise, but user code should rely on the stated ToyOS contract, not infer a permanent promise from one assembly version.

`iretq` restores user RIP, CS, RFLAGS, RSP, and SS. It performs descriptor and canonical-address checks as part of return. ToyOS returns to the unchanged saved continuation; it does not allow the dispatcher to supply arbitrary return RIP or flags.

`SYS_EXIT` is exceptional: it prints results and exits QEMU rather than returning to user space.

5. Why this is explicitly not `SYSCALL`/`SYSRET`

The x86-64 `SYSCALL` instruction does not consult an IDT gate and does not automatically switch to a kernel stack. It derives entry selectors and RIP from model-specific registers, saves return RIP in `RCX`, saves flags in `R11`, masks selected flags, and otherwise leaves software to establish safe per-CPU kernel state.

A production entry path normally needs a trusted per-CPU anchor, often reached with `swapgs`, before replacing the untrusted user RSP with a kernel stack. It must preserve state, handle tracing and scheduling, and validate the return context. `SYSRET` has architectural constraints around canonical addresses, selectors, and flags; robust kernels may choose `iretq` for cases that cannot safely use the fast return.

ToyOS implements none of that MSR/per-CPU entry machinery. Claiming that its `int 0x80` path “uses SYSCALL” because both provide syscalls would be factually wrong. The project deliberately chooses a DPL3 interrupt gate because the CPU then performs the validated TSS stack transition already supported by the kernel. It trades speed for a smaller, inspectable boundary.

Two mechanisms with different contracts

Calling both "a syscall" does not make their entry machinery interchangeable.

	ToyOS Week 8 int 0x80 / iretq	**x86-64 fast path** SYSCALL / SYSRET
Entry target	IDT[128], DPL3 gate	MSR-programmed RIP/selectors
Kernel stack	CPU loads TSS RSP0	software must replace user RSP
Saved return state	hardware stack frame: SS/RSP/RFLAGS/CS/RIP	RCX gets RIP; R11 gets flags; no automatic full stack frame
Interrupt flags	interrupt gate clears IF	MSR flag mask controls entry
Return	restore frame; iretq	SYSRET or validated fallback

Week 8 deliberately chooses the interrupt-gate path.

It reuses validated TSS stack entry for a small inspectable boundary.

Figure 2. `int 0x80` and `SYSCALL` have different entry, stack, saved-state, and return contracts; Week 8 implements only the former.

6. Diagnose an unknown syscall

The `EXPECT_FAULT=7` profile does not execute ring-3 `int 0x80`. In `process_start`, it creates a static zeroed `interrupt_frame`, sets `frame.rax = UINT64_MAX`, calls `syscall_dispatch` directly, and asserts the signed result is `-38`. It emits `EXPECTED-FAULT: syscall PASS` and exits through QEMU code `0x16`, which the host observes as status 45.

This test isolates dispatch policy. It does not test gate entry, TSS stack switching, frame assembly, or `iretq`. The normal profile supplies dynamic coverage of the integrated path and prints `SELFTEST: ring3/TSS/int80 syscalls PASS`.

That distinction is an excellent report pattern:

```
source says: default result is -38
fault profile observes: direct dispatcher rejects UINT64_MAX
normal profile observes: ring-3 calls complete
we infer: selected entry and policy contracts compose for the canonical image
```

7. Debugging checkpoints

At `syscall_dispatch`, inspect the frame using the exact `build/week8/toyos.elf`. Predict that the first call has `RAX=1`, `RDI=1`, `RSI` pointing into a user segment, and `RDX` equal to the hello string length. Inspect saved `CS`, `SS`, `RIP`, and `RSP`. Compare live `RSP` with the process kernel-stack range.

Disassemble `syscall_int_stub` and `isr_common` to verify push/pop symmetry. A source struct alone cannot prove the emitted offsets; disassembly plus debug information ties the contract to the exact artifact.

Practical challenge

Use `challenges/day-004-syscall-path.sh` from the repository root.

Purpose: connect gate policy, register ABI, frame ownership, dispatch, and return.

Predict: list the first syscall registers, the normal and unknown-call markers, host statuses 33 and 45, and why `RAX` changes on return.

Evidence: the wrapper runs both checked profiles and inspects source, symbols, and exact disassembly.

Mastery prompts

Draw both stacks immediately after hardware entry and after `isr_common`. Then explain in five sentences why replacing `int $0x80` with `syscall` in the user assembly would not create a working or safe fast-syscall path.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Syscall Path

Format: shell lab

What you will do: Trace the DPL3 int 0x80 ABI, saved frame, dispatcher, and iretq return.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-004-syscall-path.sh
```

- View lab source on GitHub

Day 5 — Safe User Copying, Pointer Validation, and Bad Pointers

Target time: approximately 3 hours

> A user pointer is a number supplied by an adversarial caller. Validate the translation and permission for every page before using the bytes.

1. Reject the tempting direct dereference

At syscall entry the kernel runs at CPL0. If it directly dereferences a user register as a C pointer, the CPU applies supervisor access rules. That may allow access to supervisor mappings the user could never touch, fault halfway through a range, or create a confused validation/use sequence.

ToyOS instead routes syscall buffers through `copy_from_user` and `copy_to_user` in `projects/toyos/src/memory.c`. These routines treat the address as an integer in the active process address space and walk its page tables explicitly.

Safe copying is both architecture and policy:

- architecture defines page-table translation and U/S/write bits;
- the helper ABI defines success/failure and direction;
- ToyOS policy caps every helper operation at 4096 bytes;
- callers decide which syscall lengths and descriptors are meaningful.

2. Walk all levels with user permission

`user_translate` starts from `active_root` and uses shifts 39, 30, 21, and 12. At every level it requires both `PTE_PRESENT` and `PTE_USER`. For a kernel-to-user copy, it also requires `PTE_WRITE`. It rejects huge entries.

At the leaf, it combines the physical page address with the 12-bit offset and returns two values: a kernel-accessible pointer through the identity mapping and the number of contiguous bytes left in that page.

Checking U/S at every level prevents a leaf under a supervisor-only parent from being treated as valid user memory. Checking write permission for `copy_to_user` prevents the kernel helper from silently bypassing intended read-only user policy. In the canonical loader all user leaves are writable, but the helper's contract is stronger than that current mapping policy.

The walk assumes four-level paging and identity access to paging structures and data frames. It does not check canonicality before indexing. For the small accepted user range this is adequate, but a hardened walker would define address bounds and overflow behavior explicitly.

Treat the pointer as untrusted arithmetic

Validation precedes byte access and repeats at every page boundary.

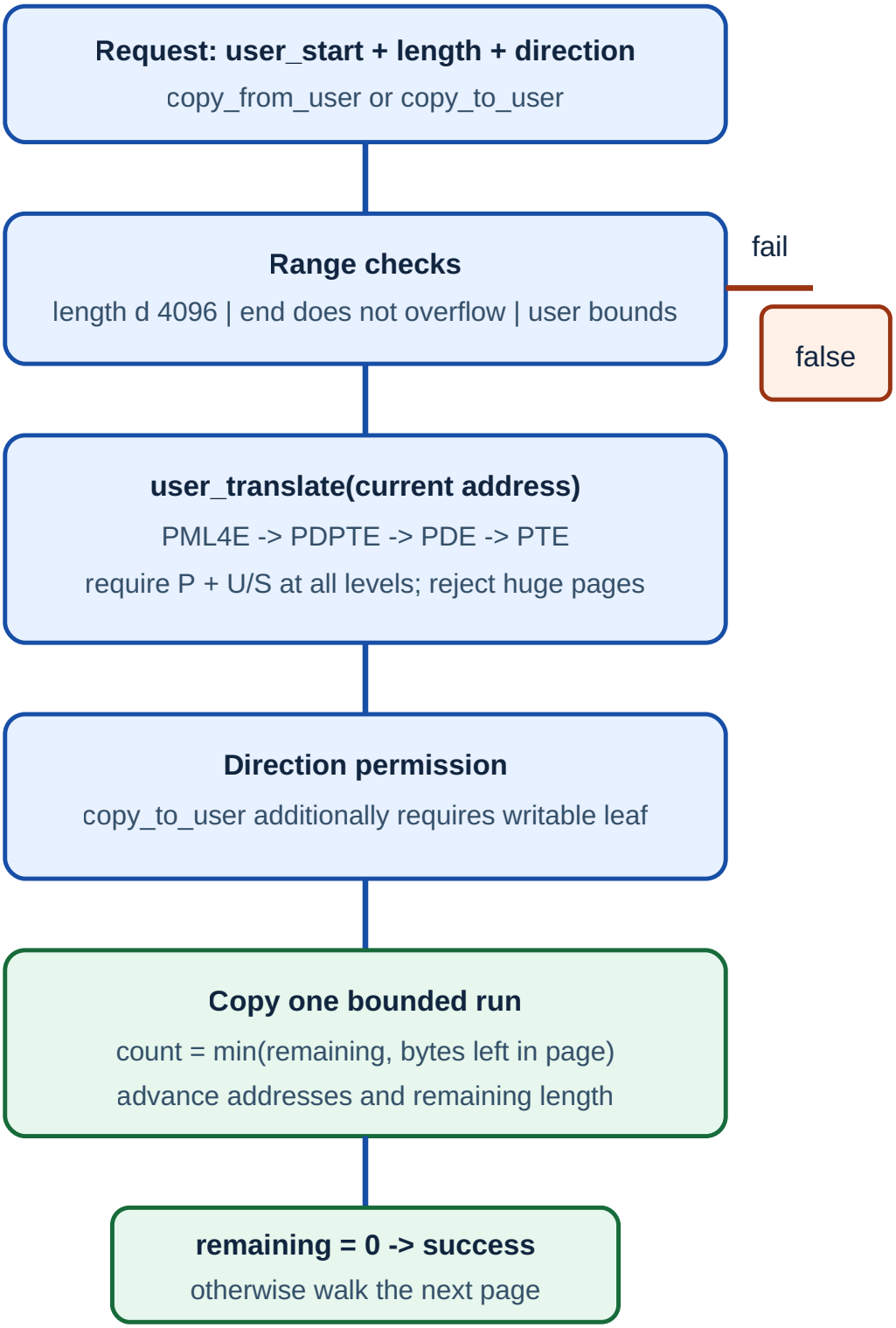


Figure 1. Safe copying combines range and overflow checks with a complete user-permitted page walk before each bounded run.

3. Copy page by page

`user_copy` loops until `length` reaches zero. For each iteration it translates the current user address, computes:

```
count = min(remaining length, bytes left in current page)
```

and copies exactly that run. It then advances the kernel pointer, user address, and remaining count.

This handles a valid buffer that crosses page boundaries. It also detects a missing or wrongly permitted second page rather than assuming the first translation covers the full range.

There is a partial-copy semantic: if an early page succeeds and a later page fails, bytes already copied remain changed, but the helper returns `false` without reporting a count. For `copy_from_user`, the kernel's temporary buffer may contain a prefix that callers must ignore on failure. For `copy_to_user`, user memory may have been partially modified. Current small canonical requests do not expose a transactional guarantee.

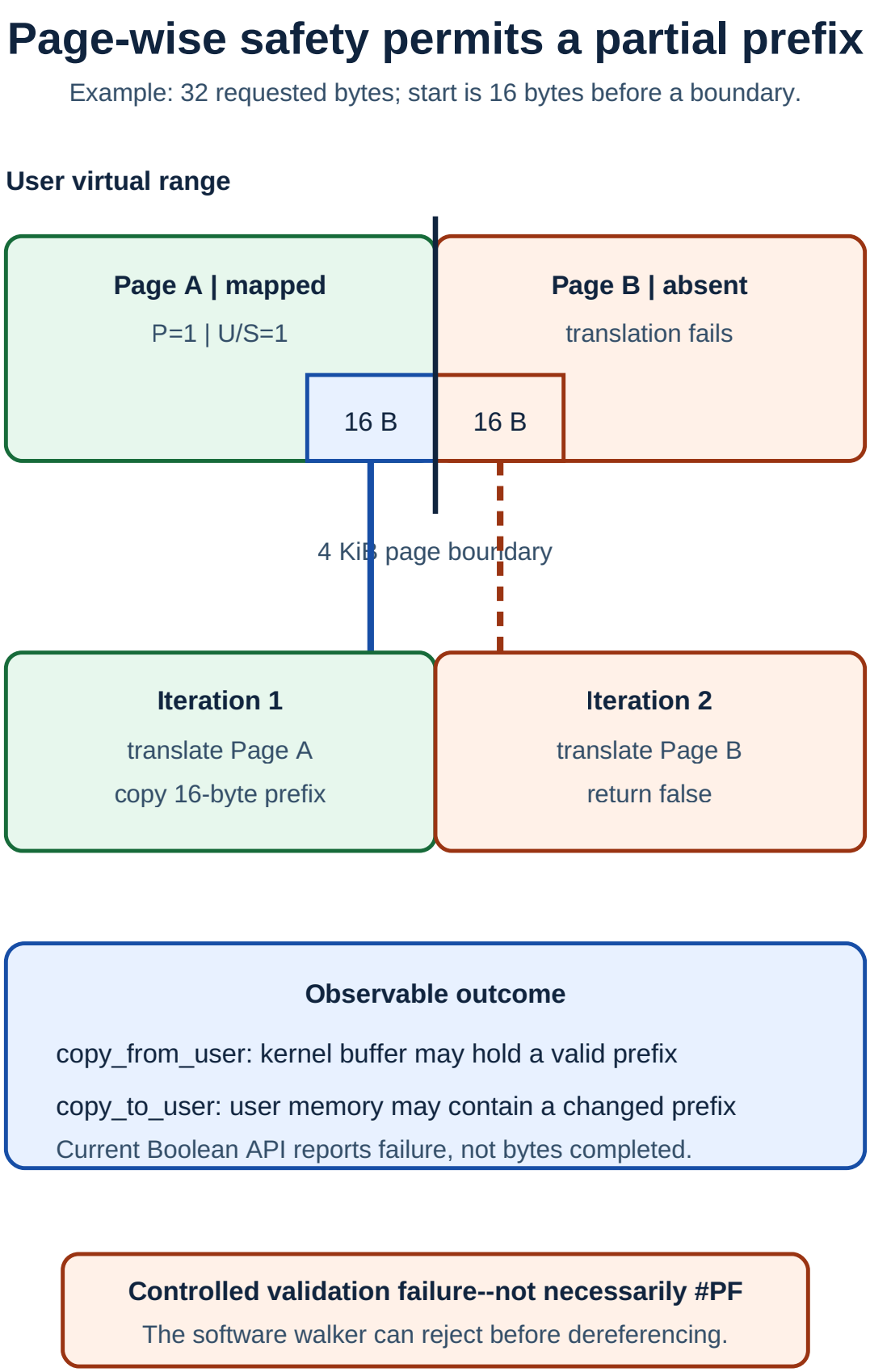


Figure 2. Crossing a fault boundary can leave a copied prefix even though the Boolean helper reports failure; the current API is not transactional.

4. Apply two independent bounds

Both public copy helpers reject lengths above 4096. `syscall_dispatch` uses a 256-byte stack buffer and independently rejects I/O lengths larger than `MAX_IO`. These bounds protect different resources:

```
4096-byte helper bound: generic user-copy work limit
256-byte syscall bound: local kernel stack-buffer capacity
64-byte pipe bound: persistent IPC storage capacity
```

Do not merge them into one “maximum syscall size.” A write to serial can use up to 256 bytes, while a write to a nearly full pipe can accept less or fail. `SYS_PIPE` copies only two 32-bit descriptors.

The dispatcher converts `frame->rdx` to `size_t`; on x86-64 both are 64 bits. It checks the bound before copying into the fixed array. That ordering is the essential memory-safety property.

5. Understand the injected bad-pointer profile

With `EXPECT_FAULT=6`, `process_start` activates the process root and then calls:

```
copy_from_user(&byte, 0x1234, 1)
```

The address is not a valid user mapping. `user_translate` returns false, `copy_from_user` returns false, and the assertion requires that rejection. The kernel emits `EXPECTED-FAULT: user-pointer PASS` and exits with debug-exit code `0x15`; the host harness expects status 43.

Despite the profile name, this is not an architectural page fault. It is an injected invalid request handled by software validation. No CPL3 instruction executes in this profile because the check happens before `enter_user`.

That behavior demonstrates a valuable boundary: invalid user input should usually become a syscall error or controlled process fault, not an unrecoverable kernel exception. ToyOS's test checks the lower-level rejection; the normal dispatcher maps copy failure to `-14`.

6. Discuss fault-aware copying honestly

Real kernels may combine range checks, access-control state, exception-table fixups, and architecture-specific user-access enable/disable mechanisms. Mappings can change concurrently, pages may be demand-faulted, and copy operations can return short counts. Spectre-class concerns and address-space separation add more constraints.

ToyOS avoids concurrency, demand paging, process teardown, and SMP. Its page tables remain stable during a syscall, and all relevant physical frames are identity mapped. Therefore a software walk followed by physical-pointer copy is sufficient for the selected model.

This does not prove resistance to malicious races, noncanonical inputs, integer wrap at the end of an address range, huge user pages, or arbitrary corrupted page tables. State these as limits rather than silently importing Linux guarantees.

7. Debug pointer failures systematically

For a failed copy, record:

1. direction and requested length;
2. user start and calculated end, including overflow analysis;
3. active CR3 and `active_root`;
4. each page-table entry and required bits;
5. page offset and cross-page count;
6. syscall result and whether any prefix changed;
7. exact profile ELF and marker/status.

If an unexpected architectural #PF occurs, add CR2, error code, saved RIP, and whether the fault happened while reading a table, touching the translated frame, or directly dereferencing a user virtual address.

The source walker is easier to reason about when written as a table:

```
level | index | entry | P | U/S | R/W | huge | next address
```

One row per level exposes the first violated invariant.

Practical challenge

Use `challenges/day-005-user-pointer.sh` from the repository root.

Purpose: verify page-wise user copying and the selected invalid-pointer rejection.

Predict: `0x1234` is rejected without an architectural #PF; the fault profile emits its marker and host status 43.

Evidence: the wrapper runs normal and injected profiles, shows walker and dispatcher contracts, and disassembles copy routines from the exact fault ELF.

Mastery prompts

Trace a 32-byte copy beginning 16 bytes before a page boundary when the second page is absent. State which bytes can change in each direction and whether the current API reports a partial count. Then explain why “the CPU will page fault if it is bad” is not an adequate syscall validation policy.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check

- View quiz source on GitHub

Put it into practice — User Pointer

Format: shell lab

What you will do: Verify bounded page-wise user copying and deterministic bad-pointer rejection.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-005-user-pointer.sh
```

- View lab source on GitHub

Day 6 — Process/FD State, Bounded Pipe IPC, and Missing Boundaries

Target time: approximately 3 hours

> IPC is not merely moving bytes. It is naming endpoints, owning buffers, defining blocking and lifetime, and deciding what happens when peers vanish.

1. Inventory the one process

ToyOS stores one static `init_process` containing:

```
root          process-owned PML4 pointer
entry         ELF entry address
user_stack    initial user RSP
kernel_stack  16 KiB supervisor allocation
pid           fixed value 1
```

The process exists for kernel lifetime. There is no state enum, parent, children, credentials, signal state, working directory, scheduling context, or teardown. `getpid` simply returns the static PID.

Week 7's two kernel tasks are not reused as a process scheduler in Week 8. For milestones 8 and 9, `kernel_main` calls `process_init` and `process_start` instead of `tasks_init` and `tasks_start`. `yield` executes only `pause` and returns zero; it does not switch to another process.

This is why “process” must be read as an ownership/isolation teaching object, not a complete lifecycle implementation.

2. Decode the descriptor convention

Descriptors are small integers interpreted by syscall policy:

- fd 1 writes to serial output;
- fd 3 reads from the global pipe;
- fd 4 writes to the global pipe.

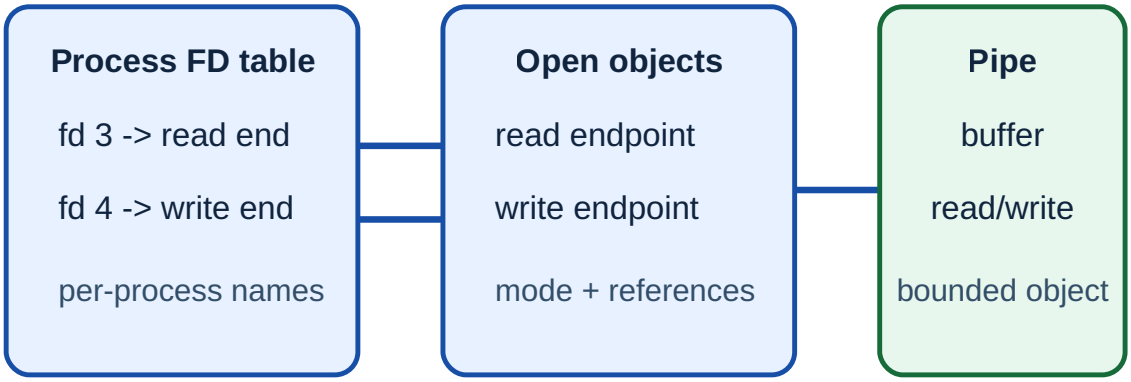
`SYS_PIPE` resets the global pipe state and copies `{3, 4}` to an eight-byte user array. There is no per-process descriptor table in Week 8, no open-file object, close operation, flags, access-mode field, reference count, or descriptor allocation algorithm.

Calling these numbers file descriptors is useful at the ABI boundary, but the implementation is a fixed convention. Ownership is global: any code able to invoke the dispatcher with fd 3 or 4 reaches the same pipe state. Since there is only one process, cross-process access control is not tested.

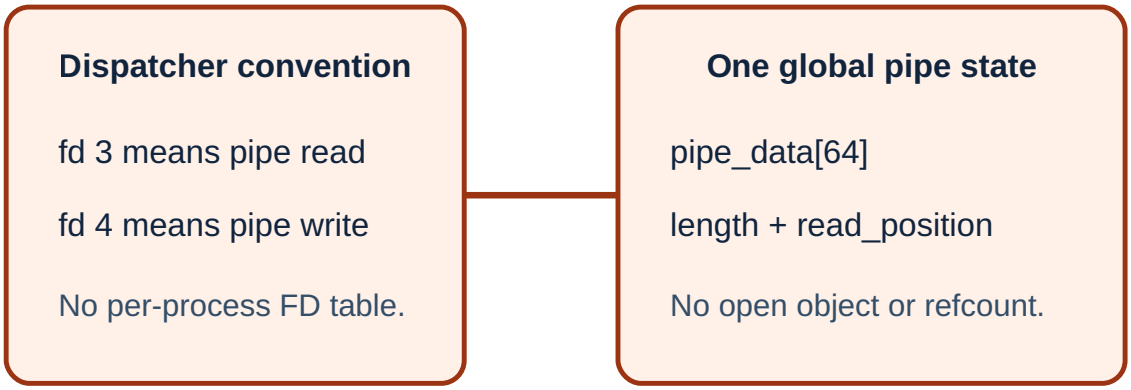
Names, endpoint objects, and pipe storage

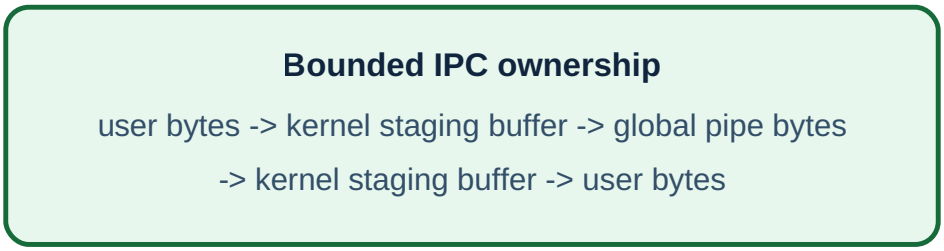
The scalable ownership model is richer than ToyOS Week 8.

General process descriptor model



ToyOS Week 8 implementation





The upper lane is a design boundary, not a claimed Week 8 feature.

Figure 1. A scalable descriptor design separates per-process names, open endpoint objects, and pipe storage; ToyOS intentionally collapses these into fixed fd policy and one global buffer.

Week 9 adds VFS descriptors for other numbers, but this week's wrappers build milestone 8 and must not import Week 9 behavior.

3. Trace the bounded pipe

The pipe state is three static objects:

```
pipe_data[64]
pipe_length
pipe_read_position
```

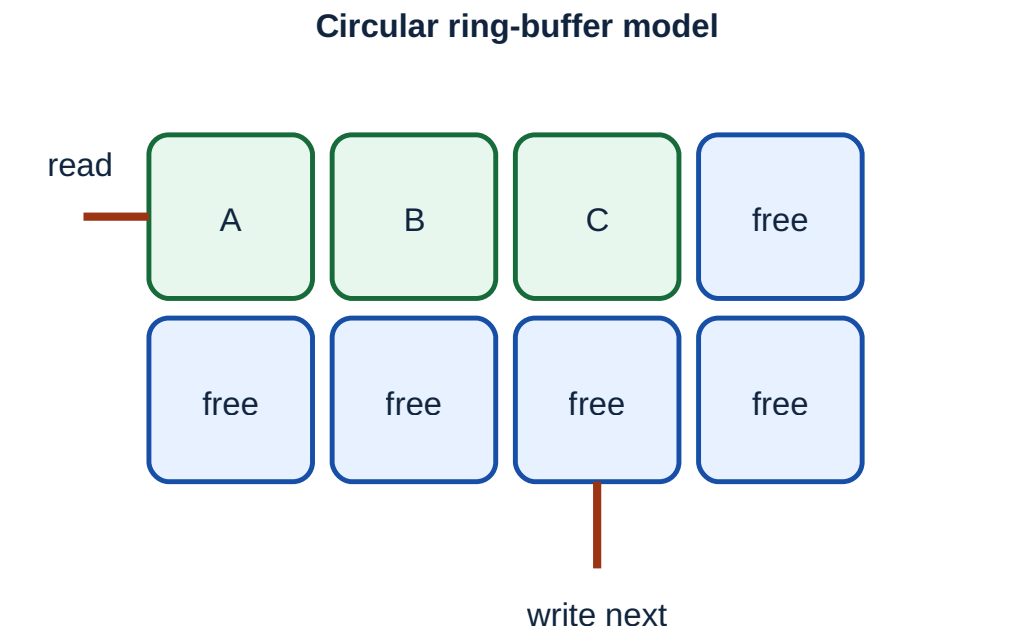
Creating a pipe sets both counters to zero. A write succeeds only when the entire request fits in `64 - pipe_length`; it appends bytes and advances `pipe_length`. It does not reuse space consumed by reads.

A read computes `available = pipe_length - pipe_read_position`, copies the smaller of requested and available, and advances `pipe_read_position`. An empty pipe therefore returns zero immediately. An over-capacity write returns `-1` immediately.

This is a bounded linear buffer, not a circular pipe. There is no blocking, wakeup, EOF, broken-pipe indication, atomic-write rule, concurrent reader, concurrent writer, or endpoint closure. Re-running `pipe` destroys previous global contents by resetting counters.

Bounded pipes need explicit state rules

Conceptual ring behavior above; exact ToyOS linear behavior below.



Empty

used = 0

read returns 0 now

Full

used = capacity

write blocks or fails by policy

ToyOS Week 8: bounded linear array, not a ring

write: append only if whole request fits

read: advance pipe_read_position; empty returns 0

consumed space is not reused; over-capacity returns -1

Capacity is 64 bytes; there is no blocking, wakeup, EOF, or close.

Figure 2. Ring buffers make reusable read/write positions explicit; ToyOS Week 8 instead uses a 64-byte linear buffer whose consumed space is not reused.

4. Follow the canonical user round trip

The embedded user program:

1. invokes `pipe(&pipe_fds)`;
2. loads fd 4 from user memory;
3. writes `"USER: bounded pipe round-trip\n"`;
4. loads fd 3;
5. reads the same number of bytes into `io_buffer`;
6. uses the returned count as serial-write length.

The kernel never directly uses those user pointers. `SYS_PIPE` uses `copy_to_user`; `SYS_WRITE` uses `copy_from_user` before choosing serial or pipe policy; `SYS_READ` fills a kernel stack buffer and then uses `copy_to_user`.

This staging gives the 256-byte local buffer temporary ownership of syscall payloads. The pipe array owns appended bytes until reset or kernel exit. `io_buffer` is process-image memory and remains owned by the process.

The normal marker `SELFTEST: bounded user copy and pipe PASS` is emitted by the exit syscall after the user program reaches its end. It proves the canonical sequence completed. It does not independently compare every byte inside the kernel; visible echoed text and control flow provide supporting evidence.

5. Place shared memory at the correct boundary

Shared-memory IPC would map the same physical frames into two or more process address spaces, with explicit lifetime and synchronization. ToyOS has only one user address space and provides no shared-memory syscall. Its copied kernel mappings are shared implementation mappings, not user IPC.

A credible shared-memory design would need to answer:

- who allocates and names the object;
- which process mappings hold references;
- when physical frames can be reclaimed;
- what user permissions each mapping has;
- how peers synchronize data and ownership;
- what happens across fork, exec, exit, or unmap.

The current page mapper and frame allocator demonstrate ingredients, but the policy and lifecycle are absent. Do not claim the pipe is shared memory: it copies through a kernel-owned array.

6. Place signals at the correct boundary

Signals require asynchronous event state and a safe user-return modification. A kernel may need pending/masked sets, dispositions, user signal frames, alternate stacks, restart semantics, process/thread targeting, and a validated trampoline or return syscall.

ToyOS has none of those. The interrupt frame proves that the kernel can see and restore user continuation state, but mutating that frame into a signal delivery is not implemented. `SYS_EXIT` terminates the VM, not one process. Unexpected user exceptions panic the kernel under current policy rather than becoming per-process signals.

The useful conceptual boundary is:

```
synchronous syscall: user intentionally enters with an ABI request
exception: CPU reports a fault tied to an instruction
signal: kernel policy schedules an asynchronous/synchronous notification
```

Architecture supplies entry and saved state. Signal semantics are operating system policy and user ABI.

7. Compare pipe and descriptor ownership

xv6 has per-process descriptor tables referring to reference-counted file objects; pipes have lock-protected buffers, sleep/wakeup behavior, and endpoint closure. Linux adds scalable descriptor tables, open-file descriptions, polling, nonblocking modes, capacity policy, namespaces, permissions, and many IPC families. Both comparisons require source revision checks.

ToyOS's fixed descriptors make the data path visible without hiding it behind large abstractions. Its limits are exactly the missing ownership questions: no endpoint object, no refcount, no close, no peer lifetime, and no wait queue.

Practical challenge

Use `challenges/day-006-pipe-ipc.sh` from the repository root.

Purpose: trace descriptor interpretation, user-copy staging, and bounded pipe state through the canonical program.

Predict: descriptors are `{3,4}`, the message fits within 64 bytes, empty reads are short, over-capacity writes fail atomically, and normal status is 33.

Evidence: the wrapper checks the integrated marker, inspects source and user disassembly, and reports exact limits.

Mastery prompts

Build an ownership timeline for the pipe message from user `.rodata` to the kernel stack buffer, global pipe, second kernel buffer, user `.data`, and serial device. Then specify the minimum new state required for two processes, blocking reads, close/EOF, shared memory, and signals.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check
- View quiz source on GitHub

Put it into practice — Pipe Ipc

Format: shell lab

What you will do: Trace fixed descriptors and the bounded global pipe through the user program.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-006-pipe-ipc.sh
```

- View lab source on GitHub

Day 7 — User→Kernel→User Trace, Fault Report, and Comparison

Target time: approximately 3 hours

> A trustworthy trace distinguishes bytes in source, instructions in an ELF, events observed in QEMU, and conclusions inferred from their agreement.

1. Build one integrated timeline

Begin at `_user_start` in the exact `build/week8/user.elf`. For the first write syscall, record:

```
user RIP prepares RAX=1, RDI=1, RSI=&hello, RDX=length
int 0x80 selects IDT[128]
CPU checks DPL3, loads TSS RSP0, saves user return state
stub adds vector/error and saves GPRs
interrupt_handler recognizes vector 128
syscall_dispatch bounds length and walks user mappings
bytes move into kernel stack buffer and then COM1
saved frame RAX becomes byte count
stub restores registers and iretq restores CPL3 continuation
```

At each event add columns for CPL, active CR3, live stack owner, saved-frame owner, buffer owner, interrupt state, and evidence type. This prevents a common mistake: saying the CPU “calls the C syscall function.” The CPU enters an assembly gate; software constructs the C boundary.

Trace claims to the evidence that supports them

No single artifact proves the complete user->kernel->user path.

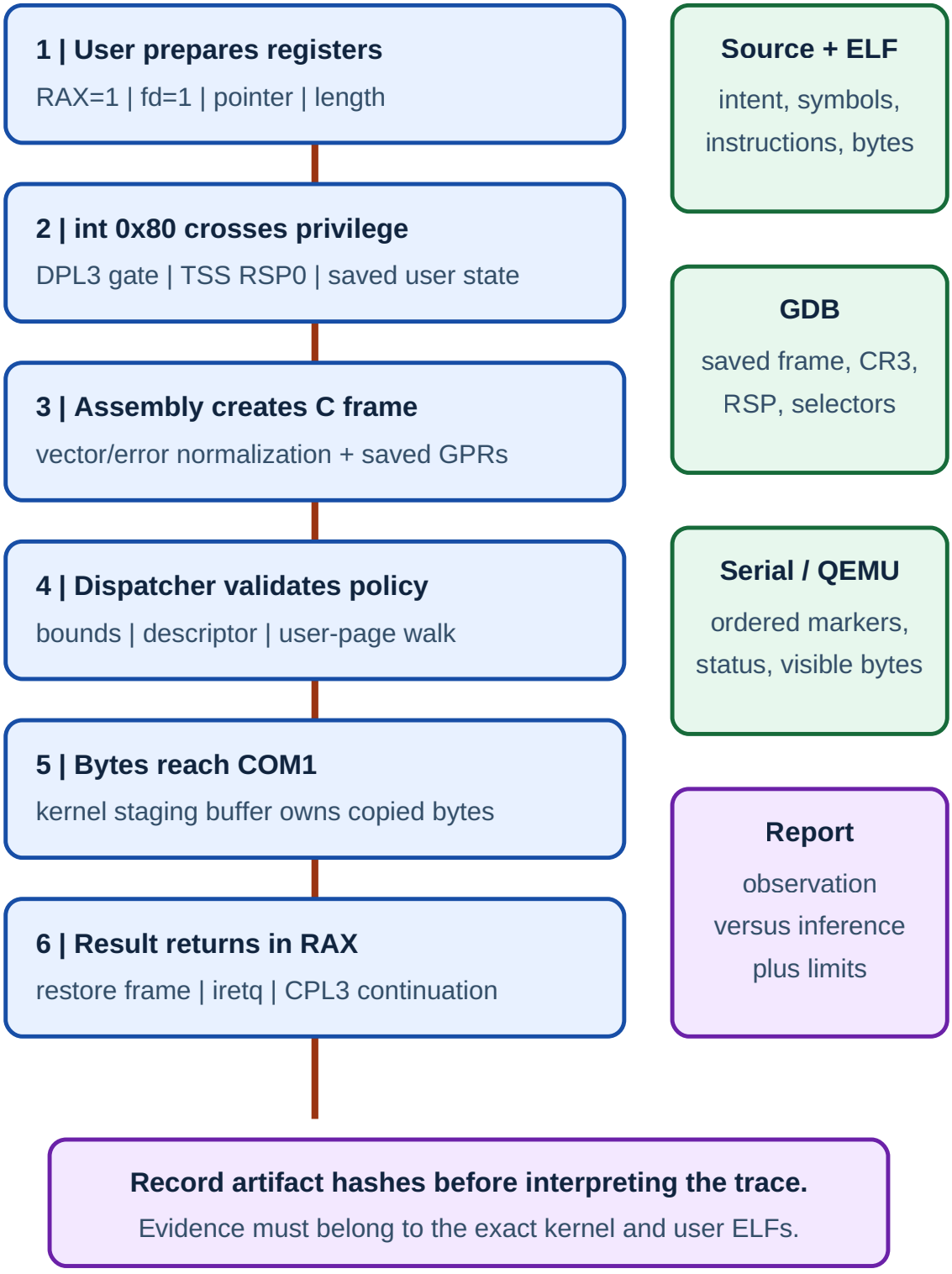


Figure 1. Each trace event needs the evidence type that can actually support it, from source and ELF bytes through saved-state observations and QEMU output.

Repeat the trace for `pipe`, pipe `write`, pipe `read`, and `exit`. Note that `exit` never returns and QEMU termination substitutes for process teardown.

2. Record artifact identity and dynamic contracts

An evidence report should include:

1. tool versions;
2. SHA-256 for kernel ELF, user ELF, and both fault-profile kernel ELFs;
3. ELF class, machine, and entry points;
4. ordered symbols for entry, loading, copying, dispatch, and return;
5. the complete `make test-week8` output;
6. source excerpts for each claimed invariant;
7. a trace with ownership and privilege;
8. interpretations and limits.

The normal profile is accepted only when the harness observes host status 33 and `WEEK8: PASS`. Because `isa-debug-exit` maps a guest byte to an odd host status, do not call 33 the kernel's exit code; the kernel writes `0x10`.

Likewise, the user-pointer profile writes `0x15` and appears as 43; the unknown-syscall profile writes `0x16` and appears as 45.

3. Diagnose by detection boundary

Invalid user pointer (`EXPECT_FAULT=6`). After activating the process root, the kernel asks `copy_from_user` to read one byte at `0x1234`. `user_translate` finds no complete present/user path, returns false, and the profile checks that result. It emits `EXPECTED-FAULT: user-pointer PASS`. This is software rejection before user entry, not vector 14.

Unknown syscall (`EXPECT_FAULT=7`). Before user entry, the kernel creates a synthetic frame with `RAX=UINT64_MAX`, calls `syscall_dispatch`, and asserts that the signed return is `-38`. It emits `EXPECTED-FAULT: syscall PASS`. This tests dispatch policy directly, not the IDT/TSS path.

Unexpected user exception. There is no selected Week 8 profile that turns a user fault into process termination or a signal. The generic handler prints the exception and panics. Therefore a debugging report should propose CR2, error code, saved RIP/CS/RSP/SS, CR3, and exact ELF as next evidence, while stating that recovery policy is absent.

These boundaries are intentionally different:

```
user-pointer validity → page-table software walker
syscall-number validity → dispatcher default case
architectural memory fault → exception handler
```

4. Use GDB without inventing observations

The canonical `gdb/week8.gdb` loads `build/week8/toyos.elf`, connects to QEMU's `:1234` stub, and stops at `kernel_main`, `process_init`, and `syscall_dispatch`. Before launching, write predictions for:

- `init_process.root` and hardware `CR3`;

- aligned TSS `RSP0` and live kernel `RSP`;
- first frame's syscall registers;
- saved ring-3 selectors;
- saved RIP immediately after `int $0x80`;
- the page-table chain for `RSI`;
- returned `RAX`.

Interactive output belongs in a report only if actually observed. The generated wrapper supplies source, artifact, and automated QEMU evidence; it does not pretend to be a live debugging transcript.

If the breakpoint cannot decode a frame, first verify artifact/profile identity and structure offsets. If execution faults before dispatch, inspect the gate, TSS, stack mapping, and selector frame rather than changing syscall policy. Debug from the earliest violated invariant.

5. Compare ToyOS with xv6-riscv

xv6-riscv has multiple processes, per-process page tables, trapframes, trampoline code, a scheduler, `fork`, `exec`, `wait`, descriptor tables, reference-counted files, and pipes with synchronization and sleep/wakeup. Its syscall instruction is `ecall`; supervisor trap CSRs and RISC-V page tables replace x86 IDT/TSS/`iretq` mechanisms.

Useful current-tree landmarks commonly include:

- `kernel/proc.c` for process lifecycle;
- `kernel/vm.c` and `kernel/exec.c` for address spaces and loading;
- `kernel/trampoline.S` and `kernel/trap.c` for user/kernel transitions;
- `kernel/syscall.c` and `kernel/sysfile.c` for dispatch;
- `kernel/pipe.c`, `kernel/file.c`, and `kernel/fdtable`-equivalent logic for descriptors and pipe lifetime.

Verify names against the revision used. xv6's simplicity is still materially richer than ToyOS's one static process.

6. Compare ToyOS with Linux

Linux separates x86 entry mechanism under `arch/x86/entry/` from generic syscall, VFS, memory-management, IPC, and signal policy. It supports both architecture fast paths and carefully validated fallback return paths, per-CPU entry state, speculation mitigations, preemption, tracing, seccomp, credentials, namespaces, and compatibility ABIs.

Its process address spaces use VMAs, demand paging, copy-on-write, ASLR, permission enforcement, teardown, and SMP TLB synchronization. User copying uses architecture-aware helpers and recoverable fault machinery rather than ToyOS's identity-mapped physical-pointer loop.

Linux descriptor state distinguishes descriptor-table entries from shared open-file descriptions and underlying objects. Pipes have lifecycle, blocking/nonblocking behavior, wakeups, polling, and capacity rules. Shared-memory families manage named or anonymous objects and mapping references. Signals maintain pending state, masks, dispositions, user frames, and validated return.

The comparison should not become a feature checklist detached from the trace. For each richer mechanism, name the invariant it addresses: per-CPU stack ownership, concurrent mapping changes, endpoint lifetime, partial-copy recovery, or asynchronous user-return modification.

eep conclusions inside the demonstrated bounda

ToyOS exposes causality; Linux manages production-scale lifecycles.

	ToyOS Week 8 selected teaching contracts	Linux richer current-kernel policy
Process and memory	one process; stable pages no teardown	threads, VMAs, faults, COW, SMP lifetime + synchronization
Syscall entry	DPL3 int 0x80 + iretq interrupts disabled in handler	x86 entry paths + mitigations per-CPU state and tracing
User copy	software page walk Boolean partial failure	architecture-aware helpers fault recovery + short counts
Descriptors and pipe	fixed fd 3/4; global 64 B linear, immediate return	FD/open-file objects; lifecycle blocking, polling, wakeups
Exit and evidence	QEMU debug-exit + markers no process reclamation	per-task exit + observability resource cleanup and accounting

Comparison question: which invariant does each feature add?

Do not import Linux guarantees into ToyOS evidence.

Figure 2. The comparison preserves ToyOS's demonstrated boundary while naming the lifecycle, concurrency, and recovery invariants Linux must add.

7. State ToyOS conclusions narrowly

Week 8 demonstrates:

- a process-owned paging root with shared supervisor kernel reachability;
- user mappings whose U/S bit propagates through the walk;
- bounded parsing/loading of the canonical embedded ELF;
- a four-page user stack and distinct kernel entry stack;
- ring-3 entry and interrupt-gate syscall round trips;
- page-wise bounded user copies;
- fixed descriptor policy and a bounded global pipe;
- controlled rejection of one bad pointer and one unknown syscall.

It does not demonstrate hostile ELF hardening, W^X, multiple processes, scheduling, blocking IPC, shared memory, signals, resource reclamation, concurrency, SMP, or production fast-syscall entry.

That final limit paragraph is part of the technical result. It tells the next engineer which assumptions can safely be reused and which must be redesigned.

Practical challenge

Use `challenges/day-007-user-kernel-report.sh` from the repository root.

Purpose: generate one temporary Week 8 report from exact artifacts, complete regression output, source contracts, integrated trace, comparison, and limits.

Predict: normal/user-pointer/syscall host statuses are 33/43/45 with their exact markers. Explain why each fault profile stops before ring 3.

Evidence: the wrapper hashes all artifacts, runs `test-week8`, inspects both ELFs, records the trace and detection boundaries, and names comparison landmarks. Add your own causal explanation; generated text is not mastery.

Integrated mastery

Explain the first serial write from ELF byte to COM1 byte. For every boundary, identify architecture, ABI, policy, ownership/lifetime, and evidence. Then choose one absent feature—second process, shared memory, or signals—and list the invariants that must be added before it can be safe.

Now check your understanding

You have finished the theory for this section. Before moving on, use the quick check below to find anything that still feels uncertain, then reinforce the idea with the practical lab when one is provided.

Quick theory check

Answer from memory first. Explanations appear only after you submit your choices.

- Start the interactive theory check

- View quiz source on GitHub

Put it into practice — User Kernel Report

Format: shell lab

What you will do: Generate a temporary Week 8 user/kernel transition and fault evidence report.

Predict the important result first, then run the lab and explain any difference between prediction and observation.

- Open the lab file

Copy and run:

```
bash 08-user-mode-syscalls-and-ipc/challenges/day-007-user-kernel-report.sh
```

- View lab source on GitHub